

Smart Blind Stick - Implementation Guide

Everything needed to build, program, and test the stick. Follow the steps in order. **No soldering required** - this is a breadboard build.

New here? Read [1_Kids_WHY.md](#) first for the "why." Shopping and prices are in [3_For_Parents_Shopping_Guide.md](#).

1. What you're building

An Arduino-powered walking stick that detects obstacles with an ultrasonic sensor and warns the user four ways:

- **Beeps** that speed up and rise in pitch as the obstacle gets closer
- A spoken "**Obstacle ahead**" message
- Handle **vibration**
- A wet-floor / **puddle** alert

It also streams a live distance graph to a laptop (great demo + auto data).

2. How it works (the science)

The HC-SR04 ultrasonic sensor sends out a 40 kHz sound pulse and listens for the echo. The Arduino measures how long the echo takes and calculates the distance:

$$\text{Distance} = (\text{Speed of sound} \times \text{Time}) \div 2$$

We divide by 2 because the sound travels to the obstacle **and back**. The closer the obstacle, the shorter the echo time. The code turns that distance into faster / higher beeps, vibration, voice, and the live graph.

3. Parts and pin map

Full shopping list and prices: [3_For_Parents_Shopping_Guide.md](#). Visual diagram: [wiring_diagram.png](#).

Every module's **VCC** -> **5V** and **GND** -> **GND** on the Arduino. Signal pins:

Component	Module pin	-> Arduino pin
HC-SR04	VCC / Trig / Echo / GND	5V / D9 / D10 / GND
Passive buzzer	S(+) / -	D4 / GND
Vibration motor	S(+) / -	D5 / GND
Warning LED	+(via 220 ohm) / -	D6 / GND
ISD1820 voice	P-L / VCC / GND	D7 / 5V / GND
Water sensor	AO / VCC / GND	A0 / 5V / GND

4. Step-by-step build (breadboard, no solder)

Build the **core first** (steps 1-6), get it working, then add voice and water.

1. **Power rails:** connect Arduino **5V** to the breadboard's red (+) rail and **GND** to the blue (-) rail. Now any component can tap power from the rails.
2. **HC-SR04:** VCC -> (+) rail, GND -> (-) rail, **Trig -> D9, Echo -> D10**. Mount it facing forward at the lower-front of the stick.
3. **Passive buzzer:** + leg -> **D4**, - leg -> (-) rail.
4. **Warning LED:** long leg (+) -> one end of the **220 ohm resistor**; the resistor's other end -> **D6**; LED short leg (-) -> (-) rail. (The resistor protects the LED.)
5. **Vibration motor - two options:**
 - *Simplest (module):* signal -> **D5**, + -> 5V, - -> GND. Done.
 - *Kit parts (bare motor):* drive it through a **BC547 transistor** so the pin isn't overloaded: D5 -> 1 kohm resistor -> transistor **base**; motor between +5V and transistor **collector**; transistor **emitter** -> GND; put a **1N4007 diode** across the motor (stripe/cathode toward +5V) to protect the circuit.
6. **Power it:** plug a **USB power bank** into the Arduino's USB port. Upload the code (section 5) and test the core.
7. **(Later) ISD1820 voice:** VCC -> 5V, GND -> GND, **P-L -> D7**. Record your message (section 7), then set `USE_VOICE 1`.
8. **(Later) Rain sensor:** VCC -> 5V, GND -> GND, **AO -> A0**. Set `USE_WATER 1`.

5. Install the software and upload

1. Download the free **Arduino IDE** from arduino.cc and install it.
2. Plug the Arduino Uno into the laptop with the **USB cable** (must be a *data* cable, not charge-only).
3. In the IDE: **Tools > Board > Arduino Uno**, then **Tools > Port** and pick the port that appears (often "usbserial" or "COM3").
4. Paste the code (section 6) into a new sketch.
5. Click **Upload** (the right-arrow button). Wait for "Done uploading."
6. Open **Tools > Serial Monitor** (set baud to **9600**) to see distance values, or **Tools > Serial Plotter** to see the live graph.

 **Never used an Arduino before?** Watch this short beginner walkthrough of the board first:
<https://www.youtube.com/watch?v=fJWR7dBuc18>

6. The full code

```
/* =====  
SMART BLIND STICK - Obstacle, Voice & Puddle Alert  
Science Exhibition | Theme: Future Technology & Robotics  
Board: Arduino UNO R3  
=====
```

HELLO, INVENTOR! Read this file top-to-bottom like a story. Every section has plain-English notes that explain WHAT the code does and WHY.

THE BIG IDEA

The stick has an ultrasonic sensor on the front. It works like a bat: it sends out a sound "ping", waits for the echo to bounce back, and times how long that takes. Sound travels at a known speed (about 343 metres every second), so from that time we can work out the DISTANCE to the object. Then we warn the user with beeps, a buzzing handle, a voice, and an LED.

HOW AN ARDUINO PROGRAM RUNS

Every Arduino sketch has two special functions:

- setup() -> runs ONCE when the board powers on (we get everything ready)
- loop() -> runs OVER and OVER, forever, many times every second

Think of setup() as "getting ready" and loop() as "doing the job".

PIN MAP (which wire goes where) - every module: VCC->5V, GND->GND

- HC-SR04 ultrasonic: Trig = D9 , Echo = D10
- Passive buzzer: D4
- Vibration motor: D5
- Warning LED: D6 (through a 220 ohm resistor)
- ISD1820 voice: P-L = D7
- Rain / water sensor: A0 = A0

```
===== */
```

/* =====

PART 1 - FEATURE SWITCHES

"#define" makes a NICKNAME for a value. Before the program is built, the computer swaps every nickname for its value. We use them as ON/OFF switches: 1 means ON, 0 means OFF.

TIP: start with VOICE and WATER OFF. Get the core working first, then turn them ON one at a time once those modules are wired.

```
===== */
#define USE_PASSIVE_BUZZER 1 // 1 = passive buzzer (can change pitch). 0 = active buzzer
                             (one fixed beep).
#define USE_VIBRATION      1 // the motor that makes the handle buzz
#define USE_LED            1 // the warning light
#define USE_VOICE          0 // the talking module (ISD1820). Set to 1 AFTER you wire it.
#define USE_WATER          0 // the puddle sensor. Set to 1 AFTER you wire it.

/* =====
PART 2 – PINS (giving each wire a name)
-----
The Arduino has many metal pins. Each has a number (like 9) or an analog
name (like A0). Instead of remembering numbers, we name each pin ONCE here.
"const int" means "a whole number that never changes". If you ever move a
wire, you only edit it in this one place.
===== */
const int PIN_TRIG = 9; // we SEND the ping out from this pin
const int PIN_ECHO = 10; // we LISTEN for the echo coming back on this pin
const int PIN_BUZZ = 4; // buzzer
const int PIN_VIBE = 5; // vibration motor
const int PIN_LED = 6; // warning LED
const int PIN_VOICE = 7; // ISD1820 "P-L": a quick HIGH pulse here plays the whole message
                           once
const int PIN_WATER = A0; // water sensor (an ANALOG pin: it reads a RANGE of values, not
                           just on/off)

/* =====
PART 3 – SETTINGS YOU CAN TUNE (the "science" lives here!)
-----
These numbers decide how the stick behaves. Changing them changes the
experiment – perfect for your data table. Units:
    distance = centimetres (cm) | time = milliseconds (ms; 1000 ms = 1 s)
    pitch    = Hertz (Hz; higher number = higher-pitched sound)
===== */
const int RANGE_MAX = 150; // start warning when an object is closer than 150 cm
const int RANGE_DANGER = 20; // full alarm when closer than 20 cm
const int GAP_FAR = 600; // far away -> 600 ms between beeps (slow beeping)
const int GAP_NEAR = 120; // very near -> 120 ms between beeps (fast beeping)
const int FREQ_FAR = 1500; // far away -> 1500 Hz tone (lower pitch)
const int FREQ_NEAR = 3000; // very near -> 3000 Hz tone (higher pitch)
const int SAMPLE_MS = 60; // measure the distance every 60 ms (about 16 times a second)
const unsigned long VOICE_COOLDOWN = 4000; // wait 4000 ms (4 s) before the voice may speak
                           again
```

```

/* =====
PART 4 – MEMORY (variables that change while the stick runs)
-----
A "variable" is a labelled box that holds a value the program can change.
"unsigned long" holds a very big, never-negative number – perfect for time,
because millis() (explained later) counts up into the billions.
"bool" holds only true or false. "int" holds a whole number.
===== */

unsigned long lastSample = 0; // the moment we last measured distance
unsigned long lastBeep   = 0; // the moment we last flipped the beep on/off
unsigned long lastVoice  = 0; // the moment the voice last spoke
bool buzzOn = false;       // is the beep currently sounding?
int smoothDist = RANGE_MAX; // our steady, best guess of the distance right now
int waterDry   = 600;       // what "dry" reads like (the stick LEARNS this at startup)


/* =====
METHOD: pingOnce()
-----
WHAT IT DOES : fires ONE ultrasonic ping and returns the distance in cm.
HOW IT WORKS :
    1. A tiny 10-microsecond HIGH pulse on Trig tells the sensor "ping now!".
    2. pulseIn() measures how many MICROSECONDS the Echo pin stays HIGH –
       that is exactly how long the sound took to fly out AND bounce back.
    3. Distance = speed-of-sound x time / 2.
       Sound travels 0.0343 cm per microsecond. We divide by 2 because the
       sound made a round trip (there AND back again).
RETURNS : distance in centimetres. If no echo returns in time, the path is
          clear, so we return a big number (RANGE_MAX + 50).
(A "microsecond" is one millionth of a second – sound is fast!)
===== */

int pingOnce() {
    digitalWrite(PIN_TRIG, LOW); delayMicroseconds(2); // settle the pin so the ping starts
    clean
    digitalWrite(PIN_TRIG, HIGH); delayMicroseconds(10); // the 10 us "send now" pulse
    digitalWrite(PIN_TRIG, LOW);
    long us = pulseIn(PIN_ECHO, HIGH, 12000UL); // time the echo (give up after ~2 m so we
    never freeze)

    if (us == 0) return RANGE_MAX + 50; // 0 means "no echo heard" = clear path ahead
    return (int)(us * 0.0343 / 2.0); // turn the time into a distance in cm
}


/* =====
METHOD: median3()
-----

```

WHY IT EXISTS : a single ping can be wrong (sound can glance off at an angle and give a junk number). So we ping THREE times and keep the MIDDLE value. Keeping the middle throws away a freak high or low reading. This is called a "median filter", and it makes the stick much steadier and more trustworthy. THE TRICK : add all three, then subtract the biggest and the smallest. What is left MUST be the middle one.

===== */

```
int median3() {  
    int a = pingOnce(), b = pingOnce(), c = pingOnce();  
    return a + b + c - max(a, max(b, c)) - min(a, min(b, c)); // (sum) - (biggest) - (smallest)  
    = middle  
}
```

/* =====

METHOD: setBuzzer(on, freq)

Turns the buzzer ON (at a chosen pitch) or OFF. It takes two inputs:

on = true/false (should it sound?)

freq = the pitch in Hz (only a PASSIVE buzzer can use this)

A PASSIVE buzzer can play different pitches with tone(); an ACTIVE buzzer only makes one fixed beep, so for it we just switch the pin HIGH/LOW.

The "#if USE_PASSIVE_BUZZER" lets the SAME program work for either buzzer.

===== */

```
void setBuzzer(bool on, int freq) {  
#if USE_PASSIVE_BUZZER  
    if (on) tone(PIN_BUZZ, freq); else noTone(PIN_BUZZ); // tone() makes the pitch; noTone()  
    stops it  
#else  
    digitalWrite(PIN_BUZZ, on ? HIGH : LOW); // active buzzer: simply on or off  
#endif  
}
```

/* =====

METHOD: setOutputs(on)

Makes the LED and the vibration motor FOLLOW the beep: when the stick beeps, the light lights up and the handle buzzes; when the beep stops, they stop too. One little helper keeps the main code tidy and easy to read.

===== */

```
void setOutputs(bool on) {  
#if USE_LED  
    digitalWrite(PIN_LED, on ? HIGH : LOW);  
#endif  
#if USE_VIBRATION  
    digitalWrite(PIN_VIBE, on ? HIGH : LOW);
```

```

#endif
}

/* =====
METHOD: level(d)
-----
Turns a distance into a simple "danger number" for the laptop graph:
  0 = clear (far away)  1 = warning (getting close)  2 = danger (very close)
This is the line the Serial Plotter draws, so judges can SEE the alert rise.
===== */
int level(int d) {
  if (d <= RANGE_DANGER) return 2;  // very close
  if (d <  RANGE_MAX)    return 1;  // in the warning zone
  return 0;               // all clear
}

/* =====
setup() - runs ONCE at power-on
-----
Here we tell each pin whether it is an OUTPUT (we control it, like the
buzzer) or an INPUT (we read it, like the Echo pin). Then we do a quick
self-test so we KNOW every part works before the exhibition starts.
===== */
void setup() {
  pinMode(PIN_TRIG, OUTPUT);  // we send pings out -> OUTPUT
  pinMode(PIN_ECHO, INPUT);   // we listen for echo -> INPUT
  pinMode(PIN_BUZZ, OUTPUT);
#ifdef USE_VIBRATION
  pinMode(PIN_VIBE, OUTPUT);
#endif
#ifdef USE_LED
  pinMode(PIN_LED, OUTPUT);
#endif
#ifdef USE_VOICE
  pinMode(PIN_VOICE, OUTPUT); digitalWrite(PIN_VOICE, LOW); // start with the voice silent
#endif
  Serial.begin(9600);         // open a "chat line" to the laptop at speed 9600

  // --- Boot self-test: one chirp + flash proves the buzzer, LED and motor work ---
  setBuzzer(true, 2500); setOutputs(true); delay(250);  // everything ON for a quarter
  second...
  setBuzzer(false, 0);  setOutputs(false);              // ...then everything OFF

#ifdef USE_WATER
  // Learn what "dry" looks like: average 20 quick readings of the water sensor.

```

```

    long s = 0;
    for (int i = 0; i < 20; i++) { s += analogRead(PIN_WATER); delay(5); }
    waterDry = s / 20;           // store the dry baseline so later we can notice "wetter than
                                // dry"
#endif

    Serial.println("distance_cm,alert_level"); // column titles for the laptop graph /
                                              // spreadsheet
}

/* =====
   loop() - runs FOREVER, many times per second
   =====
   BIG IDEA - "non-blocking":
   We never use delay() to pause in here. delay() would make the stick STOP and
   stare at the wall, missing anything that moves. Instead we check the clock
   (millis() = the number of ms since power-on) and only act when enough time
   has passed. That way the stick keeps sensing NON-STOP. This is the secret
   that makes it feel fast and responsive.

   The loop does three jobs each time:
   (A) measure the distance      (B) check for a puddle      (C) give the alert
   ===== */
void loop() {
    unsigned long now = millis(); // "now" = how many milliseconds since the stick turned on

    // ---- (A) MEASURE the distance every SAMPLE_MS, then smooth it ----
    if (now - lastSample >= SAMPLE_MS) { // has it been 60 ms since the last measurement?
        lastSample = now;                // remember this moment for next time
        int raw = median3();             // get a clean distance (the middle of 3 pings)
        // Smoothing: keep 70% of the old value + 30% of the new one. This stops the
        // number from jumping around - it eases gently toward each new reading.
        smoothDist = (smoothDist * 7 + raw * 3) / 10;
        // Send the numbers to the laptop (Serial Plotter draws them as a live graph):
        Serial.print(smoothDist); Serial.print(","); Serial.println(level(smoothDist));
    }

    // ---- (B) PUDDLE CHECK: if the floor is wet, give a special warble ----
    #if USE_WATER
        if (analogRead(PIN_WATER) < (waterDry - 150)) { // water makes the reading DROP below
            "dry"
            static unsigned long wt = 0; // "static" = this box keeps its value between loops
            static bool ws = false;
            if (now - wt >= 90) { wt = now; ws = !ws; setBuzzer(ws, 2200); setOutputs(ws); } // fast
            warble
            return; // skip the obstacle code this round - the puddle warning takes priority
        }
    #endif
}

```



```

#endif
// ---- (C) OBSTACLE ALERT: choose what to do based on the distance ----
if (smoothDist <= RANGE_DANGER) {
    // VERY CLOSE: solid tone + steady buzz + LED on, and SPEAK the warning.
    setBuzzer(true, FREQ_NEAR); setOutputs(true);
#if USE_VOICE
    if (now - lastVoice >= VOICE_COOLDOWN) { // only speak if 4 s have passed, so it never
        stutters
        lastVoice = now;
        digitalWrite(PIN_VOICE, HIGH); delay(60); digitalWrite(PIN_VOICE, LOW); // quick pulse
            = play once
    }
#endif
}
else if (smoothDist < RANGE_MAX) {
    // IN WARNING RANGE: beep on/off. The closer the object, the SHORTER the gap
    // (faster beeps) and the HIGHER the pitch. map() does the maths: it stretches
    // the distance (20..150 cm) onto the beep gap and onto the pitch.
    int gap = map(smoothDist, RANGE_DANGER, RANGE_MAX, GAP_NEAR, GAP_FAR);
    int freq = map(smoothDist, RANGE_DANGER, RANGE_MAX, FREQ_NEAR, FREQ_FAR);
    if (now - lastBeep >= (unsigned long)gap) { // is it time to flip the beep?
        lastBeep = now;
        buzzOn = !buzzOn; // "!" means flip: on->off, off->on
        setBuzzer(buzzOn, freq); setOutputs(buzzOn);
    }
}
else {
    // ALL CLEAR: nothing is close, so keep everything silent and off.
    setBuzzer(false, 0); setOutputs(false); buzzOn = false;
}
}

```

Why this code is better than a basic sketch:

1. **Non-blocking** (`millis()`) - never stops sensing, even while alerting.
2. **Noise filtering** (median-of-3 + smoothing) - rejects HC-SR04 spikes.
3. **Proportional alert** - beep rate and pitch scale with distance.
4. **Edge-triggered voice + cooldown** - speaks once cleanly, no stutter.
5. **Boot self-test** - one chirp + flash confirms every part works.
6. **Auto-calibrating** water threshold at startup.
7. **Live CSV streaming** for the laptop graph and the required data table.

7. Record the voice message (ISD1820)

1. Power the ISD1820 (VCC -> 5V, GND -> GND).

2. Hold the onboard **REC** button and speak "*Obstacle ahead, please stop*" into the tiny microphone (up to ~10 seconds). Release REC.
3. Press **PLAY** to check it sounds clear. Re-record if needed.
4. Wire **P-L -> D7**, set `USE_VOICE 1` in the code, and re-upload. The Arduino now triggers that message automatically when an obstacle is very close.

8. Switch on voice and water

After the core works and the modules are wired:

- Set `#define USE_VOICE 1`
- Set `#define USE_WATER 1`
- Re-upload. (The water sensor auto-calibrates to "dry" each time it powers on.)

9. Test and calibrate

- **Self-test:** on power-up you should hear one chirp + see the LED flash. That confirms the buzzer, LED, and vibration all work.
- **Walk test:** move your hand toward the sensor. Beeps should get faster and higher; under ~20 cm it goes to a solid tone (+ voice if enabled).
- **Water test:** touch a wet cloth to the rain sensor - you should hear the distinct fast warble. If it triggers in dry air, raise the `150` margin in the water line of the code.

10. Data collection (required by the school)

Run ~15-20 trials with a measuring tape and record:

Trial	Actual distance	Expected alert	Actual alert	Result
1	100 cm	Slow beep	Slow beep	OK
2	70 cm	Slow beep	Slow beep	OK
3	45 cm	Fast beep	Fast beep	OK
4	18 cm	Solid + voice	Solid + voice	OK
5	wet floor	Double-warble	Double-warble	OK

Accuracy = (successful trials / total trials) × 100% (e.g. 18/20 = 90%). Tip: the Serial Monitor prints the exact distances - copy them into Excel/Sheets to build the table and chart automatically.

11. Tuning cheat-sheet

If you want...	Change
Warn from further away	raise <code>RANGE_MAX</code> (e.g. 200)
Alarm only when very close	lower <code>RANGE_DANGER</code> (e.g. 15)
Faster / slower beeping	lower / raise <code>GAP_NEAR</code> / <code>GAP_FAR</code>
Use an active buzzer instead	set <code>USE_PASSIVE_BUZZER</code> to <code>0</code>
Add voice / water later	flip <code>USE_VOICE</code> / <code>USE_WATER</code> to <code>1</code>

12. Troubleshooting

Problem	Likely fix
No beep at all	Buzzer + leg must go to D4 ; if it's an <i>active</i> buzzer set <code>USE_PASSIVE_BUZZER 0</code>
Beeps non-stop	Trig/Echo swapped - check Trig=D9, Echo=D10 ; make sure the sensor isn't facing something within a few cm
Distance jumps around	Mount the sensor flat and facing forward; soft/cloth/angled surfaces absorb ultrasound
Voice won't play	Did you record it? Is P-L -> D7 and <code>USE_VOICE 1</code> ? Is the speaker connected?
Vibration weak / none	A bare motor needs the transistor circuit; check the transistor legs, or use the plug-and-play module
Upload fails	Select Arduino Uno + correct Port ; close the Serial Monitor during upload; use a <i>data</i> USB cable
Nothing happens after upload	Wrong board/port, or a charge-only USB cable

13. The demo that wins it (zero extra hardware)

At the booth, keep the Arduino plugged into a laptop via USB:

- **Tools > Serial Plotter** shows a live graph of distance dropping as a hand approaches, while the pitch rises - judges see and hear it together.
- **Tools > Serial Monitor** prints CSV (`distance_cm,alert_level`) you can copy straight into a spreadsheet for your data table.

14. Judge Q&A prep

- "*How does it measure distance?*" -> "It sends 40 kHz ultrasonic waves, times the echo, and uses speed of sound / 2."
- "*Why vibration AND sound?*" -> "Sound is missed in noisy markets/roads; vibration in the handle still alerts the user."

- "What's your accuracy and why isn't it 100%?" -> cite your %, note that soft/angled surfaces absorb ultrasound.
- "What makes it smart, not just a buzzer?" -> "It filters sensor noise, the alert is proportional to distance, and it streams live data we graphed."

15. Reference videos

- **New to Arduino? Start here** (what the board is and how to program it, for absolute beginners): <https://www.youtube.com/watch?v=fJWR7dBuc18>
- **Closest to this build** (Arduino Uno + ultrasonic + vibration + buzzer): <https://www.youtube.com/watch?v=qtIgMRC9qCo>
- **Science-fair framing** (parts list + circuit + demo): <https://www.youtube.com/watch?v=7hmTb7lqm4s>

These show the basic version. Ours adds proportional (rising-pitch) alerts, noise filtering, voice, puddle detection, and the live data graph - use the videos for assembly ideas, then highlight our upgrades as what makes the project stand out.

Reach out to [Vikrant Singh](#) for any queries.